

Guía de Montaje: Herencia de Plantillas y Jinja2

Esta guía te ayudará a implementar el sistema de plantillas dinámicas en Flask. Sigue los pasos en orden para evitar errores de rutas.

Objetivos de la sesión

1. Comprender el **Renderizado del lado del Servidor (SSR)**.
2. Implementar **Herencia de Plantillas** para centralizar el diseño (Navbar/Footer).
3. Usar **Lógica en HTML** (bucles y condicionales) mediante Jinja2.

Momentos de la Clase

1. El Gancho - "El problema de las 100 páginas"

1. **Concepto:** Introducir **DRY** (*Don't Repeat Yourself*).
2. **Explicación visual:** Mostrar cómo un archivo base.html sirve de "esqueleto" y los demás archivos solo rellenan el "contenido".

2. Demostración Técnica - Los superpoderes de Jinja2

Explicar la sintaxis básica que verán en el código:

- `{{ variable }}`: Para imprimir algo (un nombre, un título).
- `{% comando %}`: Para lógica (herencia, bucles, condicionales).
- `{# comentario #}`: Para notas internas.

3. Práctica Guiada - Montaje del Dashboard

Los estudiantes seguirán el documento de paso a paso para:

1. Crear el archivo base.html con los bloques `{% block content %}`.
2. Crear la página de Inicio y de Servicios heredando de la base.
3. Pasar una lista de servicios desde Python y mostrarla con un ciclo `for` de Jinja2.

4. El Reto

Agreguen un condicional `if` para que, si el usuario está logueado (simulado con una variable), el menú diga 'Hola [Nombre]', de lo contrario diga 'Iniciar Sesión'.

Paso 1: Organización de la Carpeta

Para que Flask encuentre tus archivos, la estructura debe ser **exactamente** esta:

```
proyecto_clase2/
├── app.py          # El cerebro (Python)
├── templates/      # Carpeta obligatoria (en minúsculas y plural)
│   ├── base.html   # El molde general
│   ├── index.html  # La página de inicio
│   └── servicios.html # La lista de servicios
```

Paso 2: Crear el Molde Maestro (base.html)

Este archivo contiene todo lo que NO cambia (Menú, Footer, CSS).

1, Busca la etiqueta <main> o el lugar donde quieras que aparezca el contenido de cada página.

2. Escribe estas etiquetas mágicas:

```
{% block content %}
```

```
{% endblock %}
```

Todo lo que pongas entre esas etiquetas será reemplazado por el contenido de las otras páginas.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{{ titulo }} | EcoBooking</title>
  <!-- Usamos Tailwind para diseño rápido -->
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-50 flex flex-col min-h-screen">

  <!-- Navegación Global -->
```

```

<nav class="bg-indigo-600 text-white p-4 shadow-lg">
  <div class="container mx-auto flex justify-between items-center">
    <h1 class="text-xl font-bold"><img alt="EcoBooking logo" data-bbox="450 145 475 165"/> EcoBooking</h1>
    <div class="space-x-4">
      <a href="/" class="hover:text-indigo-200">Inicio</a>
      <a href="/servicios" class="hover:text-indigo-200">Servicios</a>
    </div>
  </div>
</nav>

<!-- Contenido Dinámico -->
<main class="container mx-auto mt-10 p-4 flex-grow">
  {% block content %}

  <!-- Aquí se inyectará el código de las otras páginas -->

  {% endblock %}
</main>

<!-- Pie de Página Global -->
<footer class="bg-gray-800 text-gray-400 text-center p-6 mt-10">
  <p>&copy; 2024 Diseño Web II - Aprendiendo Jinja2</p>
</footer>

</body>
</html>

```

Paso 3: Crear las Páginas Hijas (index.html y servicios.html)

En estos archivos **NO** vuelvas a escribir <html>, <head> ni <body>.

1. Al principio del archivo, avísale a Python que vas a usar el molde:

```
{% extends "base.html" %}
```

2. Abre el bloque de contenido:
{% block content %}
3. Escribe tu HTML (títulos, textos, etc.).
4. Cierra el bloque:
{% endblock %}

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<div class="text-center py-20">
```

```
<h1 class="text-5xl font-extrabold text-gray-800 mb-4">Bienvenido a la Era Dinámica</h1>
```

```
<p class="text-xl text-gray-600">Este contenido vive dentro de un bloque de Jinja2.</p>
```

```
<a href="/servicios" class="inline-block mt-8 bg-indigo-600 text-white px-6 py-3 rounded-lg  
hover:bg-indigo-700">
```

```
Ver Servicios Disponibles
```

```
</a>
```

```
</div>
```

```
{% endblock %}
```

Paso 4: Lógica Dinámica en servicios.html

Para mostrar la lista de servicios que enviamos desde Python, usamos un bucle **for**:

```
{% for s in servicios %}
```

```
<!-- Este bloque se repetirá por cada servicio en la lista -->
```

```
<h3>{{ s.nombre }}</h3>
```

```
<p>Precio: {{ s.precio }}</p>
```

```
<!-- Condicional para disponibilidad -->
```

```
{% if s.disponible %}
```

```
<p>✅ Disponible</p>
```

```
{% else %}
```

```
<p>❌ Agotado</p>
```

```
{% endif %}
```

```
{% endfor %}
```

Paso 5: Conexión en Python (app.py)

Asegúrate de que tus funciones en Python estén enviando los datos correctamente con `render_template`.

- **Para el Inicio:** `return render_template('index.html', titulo="Inicio")`
- **Para Servicios:** `return render_template('servicios.html', servicios=datos_servicios)`

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
# --- RUTAS ---
```

```
@app.route('/')
```

```
def home():
```

```
    # Pasamos una variable 'titulo' para que base.html la use en la pestaña del navegador
```

```
    return render_template('index.html', titulo="Inicio")
```

```
@app.route('/servicios')
```

```
def servicios():
```

```
    # Simulamos datos que vendrían de una base de datos
```

```
    datos_servicios = [
```

```
        {"nombre": "Corte de Cabello", "precio": 25, "disponible": True},
```

```
        {"nombre": "Barba Premium", "precio": 15, "disponible": True},
```

```
        {"nombre": "Tinte Pro", "precio": 40, "disponible": False},
```

```
        {"nombre": "Masaje Facial", "precio": 20, "disponible": True}
```

```
    ]
```

```
    return render_template('servicios.html',
```

```
        titulo="Servicios",
```

```
        servicios=datos_servicios)
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

Consejos si algo falla:

4. **Error "TemplateNotFound":** Revisa que la carpeta se llame templates y que los nombres de los archivos en Python coincidan exactamente con los archivos reales.
5. **La página sale en blanco:** Revisa que hayas escrito bien {% block content %} (con el espacio y los porcentajes) tanto en la base como en el hijo.
6. **El servidor no inicia:** Mira la terminal, si hay un error de sintaxis en Jinja2, Flask te dirá la línea exacta del HTML que tiene el problema.